



Estrutura de Dados

2026.2

Professor: João Nicollas

Problema

Trabalhamos com vetores de tamanho fixo.

Como armazenar uma lista de alunos com tamanho dinâmico?

Armazenar uma quantidade de números que pode crescer ou diminuir em tempo de execução ?

Linked List

É uma estrutura de dados linear usada para armazenar uma coleção de elementos. Os elementos de uma lista são chamados de nó, então podemos dizer que:

Uma lista ligada(**Linked List**) é uma sequência de nós (**nodes**) interligados, onde cada nó armazena seus dados e o endereço do próximo nó.

Estrutura de dados linear é uma estrutura em que os elementos de dados são organizados sequencialmente ou linearmente, onde os elementos são anexados ao seu adjacente anterior e ao próximo no que é chamado de estrutura de dados linear.

Pilha, Fila, Listas e Arrays.

Linked List

Existem 3 tipos de listas ligadas:

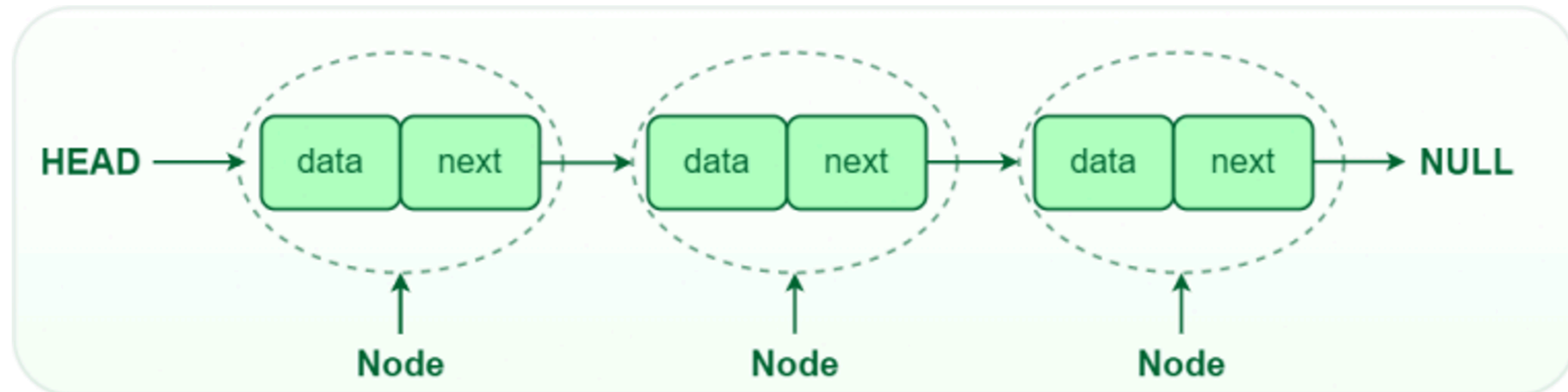
- **Lista Simplesmente encadeada**
- **Lista Duplamente encadeada**
- **Lista Circular**

Singly Linked List

Uma Lista Simplesmente Encadeada é uma estrutura onde os elementos (nós) não estão armazenados de maneira sequencial e cada elemento está conectado apenas ao próximo, através de **ponteiros**.

Singly Linked List

Uma Lista Simplesmente Encadeada é uma estrutura onde os elementos (nós) não estão armazenados de maneira sequencial e cada elemento está conectado apenas ao próximo, através de ponteiros. último elemento da lista sempre vai apontar par NULL, que indica o fim da lista.



Singly Linked List

Os nós são formados por duas partes, a data e o ponteiro. A data corresponde a informação que você deseja armazenar, pode ser um inteiro, uma string, um char, um double, e também pode ser structs. O ponteiro é o que aponta para o próximo elemento da lista.



Singly Linked List

Uma lista ligada é acessada através do nó head, que é o primeiro nó da lista. E o último nó da lista aponta para NULL, que indica o fim da lista. O último nó também é chamado de tail.

Por que listas ligadas são importantes ?

1. Estrutura de dados dinâmica: Seu tamanho cresce e diminui em tempo de execução com as operações de remoção e inserção.
2. Fácil remoção e inserção: Atualizamos apenas o endereço.
3. Utilização eficiente de memória
4. Implementação de estruturas de dados mais complexas, como pilha, fila, grafos e hash maps.

Singly Linked List

Desvantagens

1. Não tem uma boa performance para acessar os elementos, comparado aos arrays.
2. Aumento de memória: Singly linked lists precisam de mais memória pois, também armazenam os ponteiros.
3. Perca de dados: Singly linked lists estão vulneráveis a perda de dados.

Singly Linked List

Desvantagens

4. Não são adequadas para processamento paralelo: não são adequadas para processamento paralelo, pois a atualização de um nó requer acesso exclusivo ao seu próximo ponteiro, o que não pode ser feito facilmente em um ambiente paralelo.
5. Não é possível percorrer a lista de trás para frente.

Singly Linked List

Como criamos Listas Simplesmente Encadeadas em C++?
Começamos pelo Nó e depois a lista.

```
struct No {  
    int data;  
    No *next;  
};
```

```
int main() {  
  
    No * head = new No;  
  
    head -> data = 1;  
    head -> next = NULL;  
  
    No * segundoElemento = new No;  
    segundoElemento -> data = 2;  
    segundoElemento -> next = NULL;  
  
    head -> next = segundoElemento;  
  
    return 0;  
}
```

Singly Linked List – Operações

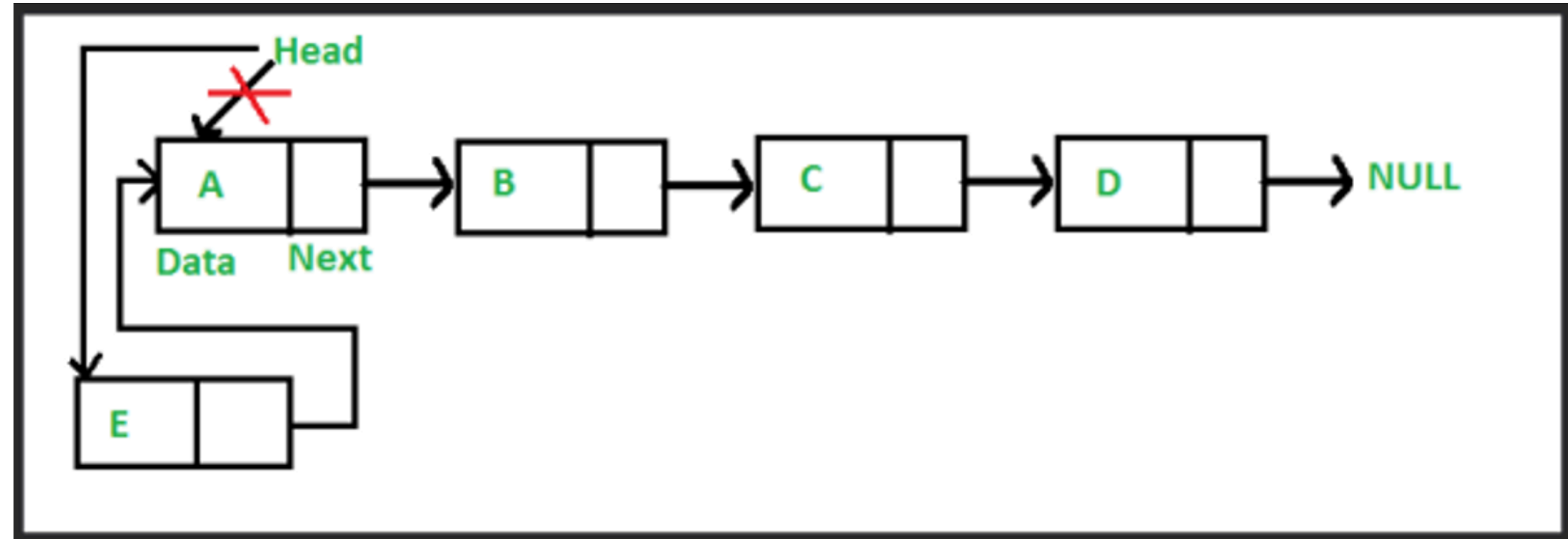
Operações Como manipular essas listas?

Existem operações básicas:

- Inserção (Inicio, n Posição, Fim)
- Remover (Inicio, n Posição, Fim)
- Tamanho (Quant. elementos)
- Imprimir
- Busca

Singly Linked List – Operações

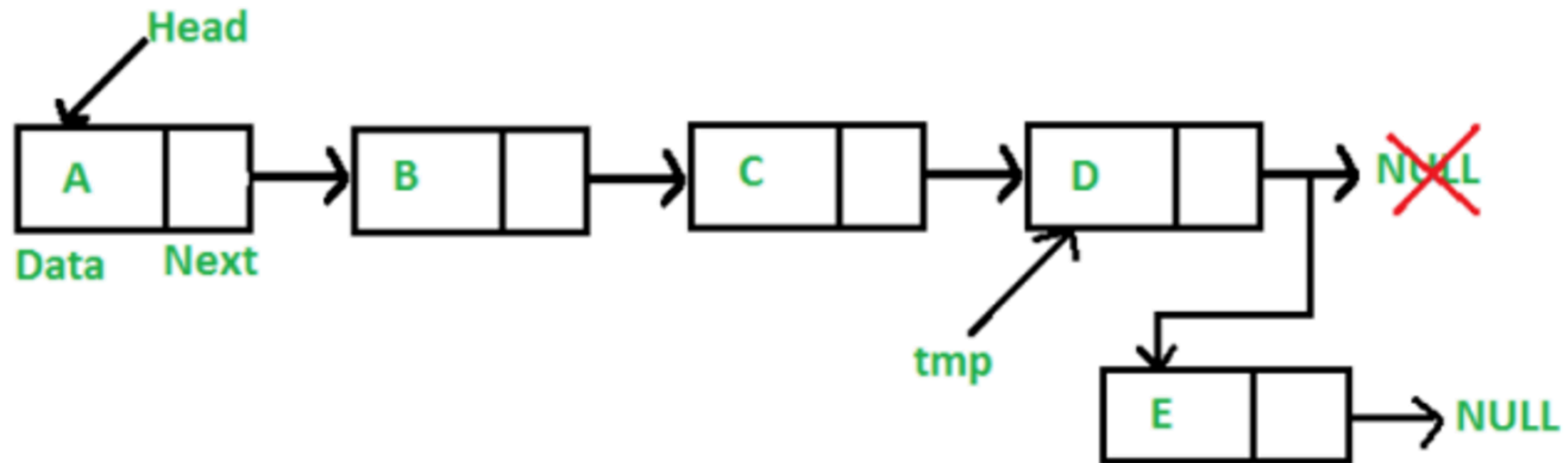
Inserção (Início)



```
21 void inserirNoComeco(No *&head, int value) {  
22     No* novoNo = new No;  
23  
24     novoNo -> data = value;  
25     novoNo -> next = head;  
26  
27     head = novoNo;  
28 }
```

Singly Linked List – Operações

Inserção (Final)



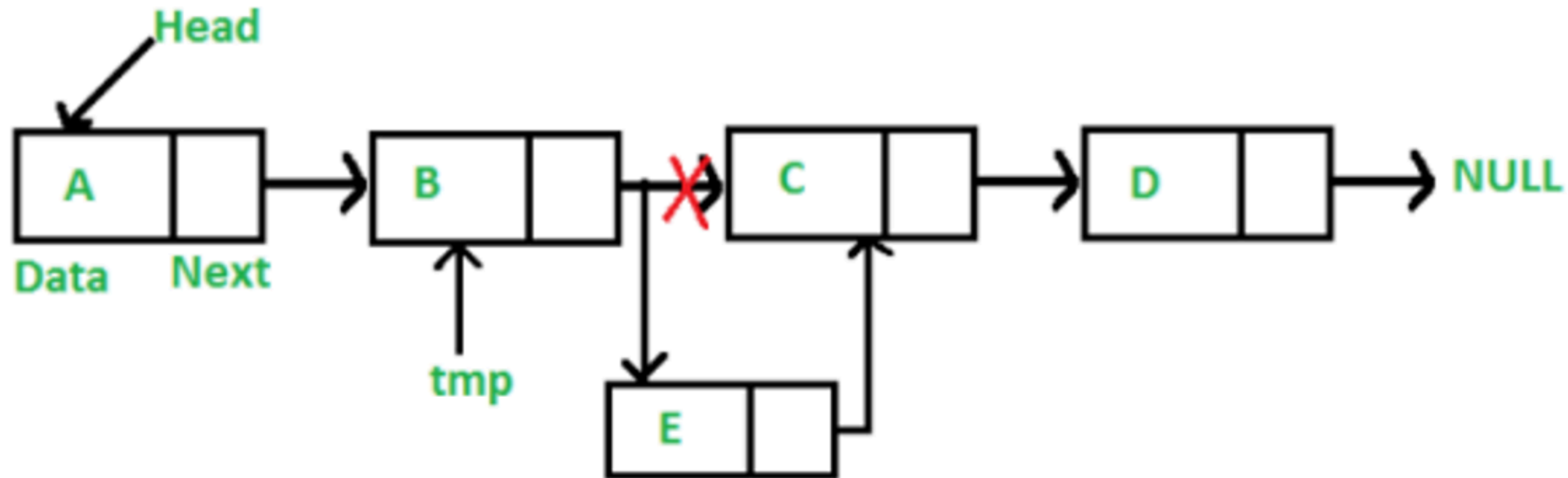
Singly Linked List – Operações

Inserção (Final)

```
void inserirNoFinal(No *&head, int value) {  
    No* novoNo = new No;  
    novoNo -> data = value;  
    novoNo -> next = NULL;  
  
    if (head == NULL) {  
        head = novoNo;  
        return;  
    }  
  
    No *aux = head;  
    while(aux -> next != NULL) {  
        aux = aux -> next;  
    }  
  
    aux -> next = novoNo;  
}
```

Singly Linked List – Operações

Inserção (Posição N)



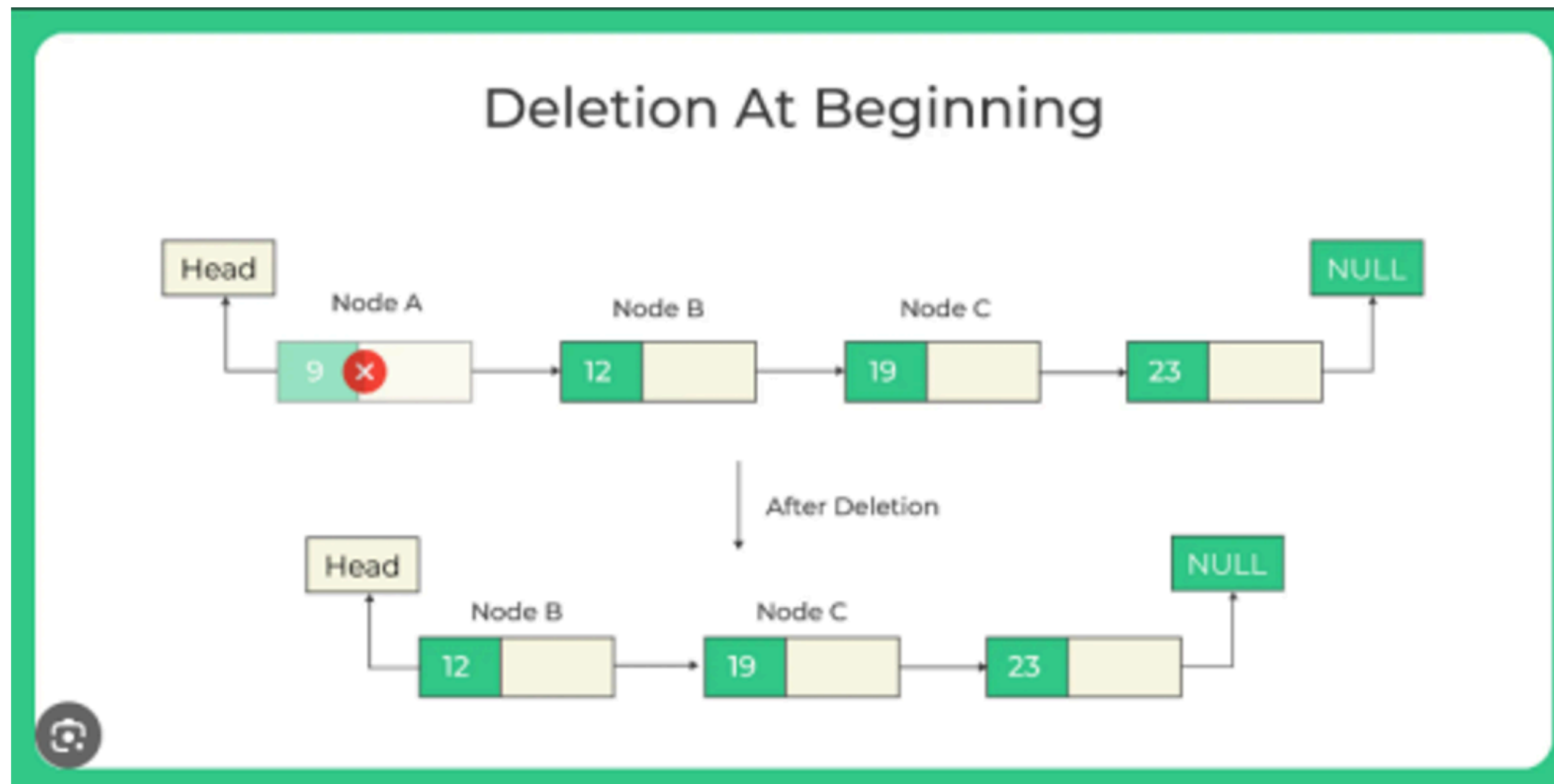
Singly Linked List – Operações

Inserção (Posição N)

```
void inserirPosicao(No *&head, int posicao, int value) {  
    No* novoNo = new No;  
    novoNo -> data = value;  
  
    if (head == NULL) {  
        head = novoNo;  
        return;  
    }  
  
    No *aux = head;  
    for(int i = 0; i < posicao - 1; i++) {  
        if(aux -> next != NULL) {  
            aux = aux -> next;  
        }  
    }  
  
    novoNo -> next = aux -> next;  
    aux -> next = novoNo;  
}
```

Singly Linked List – Operações

Remover (Inicio)



Singly Linked List – Operações

Remover (Inicio)

```
void removerNoComeco(No *&head) {  
    if (head == NULL) {  
        cout << "Linked List Vazia, nada para deletar" << endl;  
        return;  
    }  
  
    head = head -> next;  
}
```

Singly Linked List – Operações

Remover (Inicio)

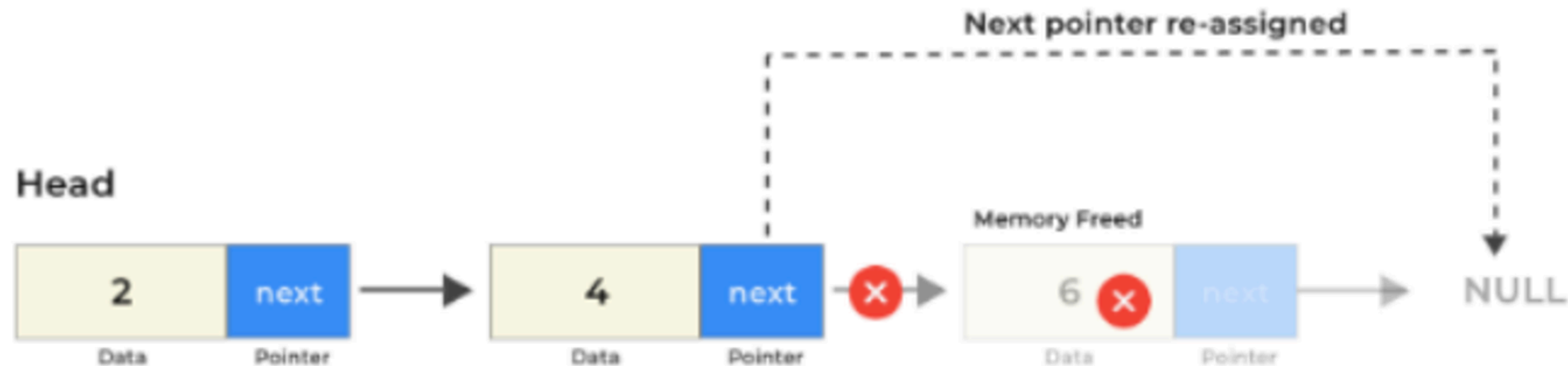
```
void removerNoComeco(No *&head) {  
    if (head == NULL) {  
        cout << "Linked List Vazia, nada para deletar" << endl;  
        return;  
    }  
  
    No* temp = head;  
    head = head -> next;  
  
    delete temp;  
}
```

Singly Linked List – Operações

Remover (Fim)

Deletion at End

2, 3 and 6 are existing nodes, deletion has to happen from End



Singly Linked List – Operações

Remover (Fim)

```
void removerNoFim(No *&head) {  
    if (head == NULL) {  
        cout << "Linked List Vazia, nada para deletar" << endl;  
        return;  
    }  
  
    No* temp = head;  
    No* anterior = NULL;  
    while(temp -> next != NULL) {  
        anterior = temp;  
        temp = temp -> next;  
    }  
  
    anterior -> next = NULL;  
    delete temp;  
}
```

Singly Linked List – Operações

Percorrer e exibir os elementos

```
void printList(No *head){  
    No *p = head;  
    printf("\n[");  
  
    while(p != NULL) {  
        printf(" %d ", p->data);  
        p = p->next;  
    }  
    printf("]");  
}
```

Singly Linked List – Operações

Implementem, os seguintes métodos:

- Tamanho
- Remover em qualquer posição